Existence Before Relevance: Same-Category Demonstrations Narrow a Prerequisite-Chain Gap in LLM Plan Generation, and Pass On Their Defects

Zixuan Wang[1] **Ziming Dai**[2] **Yu Wu**[3] **Li Ning**[4]
[1]Shenzhen College of International Education [2]BASIS International Schools
[3]CuliuTech [4]Stellaris AI Limited
zw6963470@gmail.com benjamin.dai66@gmail.com
wuyu@culiu-tech.com lining@stellaris-ai.com

Abstract

Large language models (LLMs) are increasingly used as task planners for embodied agents, yet long prerequisite chains remain a weak point. We study this in offline plan generation for a Minecraft agent, scoring each plan with a rule-based validator that walks it over a virtual inventory. For five of the seven LLMs we test (8B to 671B parameters where disclosed), plans for two-chain (L2) tasks pass only 10–30% of the time without retrieval, against 45–95% on single-material tasks. Adding three same-category demonstrations to the prompt improves all five. In a four-condition ablation on one model, randomly chosen same-category demonstrations recover most of the gain (+31 percentage points; +20 when the scorer also charges consumed materials), and ranking them by relevance over the game's recipe graph adds a smaller increment over lexical ranking (+12 points) that neither our pre-specified correction nor the stricter scorer confirms. Demonstration content is also inherited verbatim: an invalid command in the combat demonstrations appears in 71.5% of retrieval trials on those tasks and in none of 505 trials without retrieval. In this setting, adding same-category demonstrations to a shared rule text matters more than selecting them precisely, and their defects have to be checked because they are copied.

1 Introduction

An LLM that plans for a virtual embodied agent needs long chains of procedural, version-specific knowledge: which tool mines which block, what a recipe consumes, which intermediate item must exist first. A symbolic planner is handed this knowledge as a domain model; an LLM planner is assumed to have absorbed it in pretraining. Minecraft makes the difference measurable. Frontier models decompose single-step goals readily [Wang et al., 2024a, 2023], and Plancraft [Dagan et al., 2025] finds that they struggle with multi-step crafting. We ask which facet of that knowledge an LLM planner must be *given* rather than assumed to hold: its existence in the prompt as worked examples, its relevance to the query, its correctness, or the model's capacity to use it.

Scope. This is a study of LLM *plan generation* with an embodied motivation. Every confirmatory result comes from scoring generated plans offline: a rule-based validator walks each plan over a virtual inventory whose initial state is given by construction, with no perception, symbol grounding or execution. A live-execution study does not separate the conditions and is reported as a limitation, not as evidence (§6.2). The neuro-symbolic interaction we evaluate is therefore *pre-generation knowledge supply*: the `minecraft-data` recipe graph drives the ranking of the demonstrations that enter the prompt. In the static study the validator serves only as the metric (Figure 1). Every condition shares a

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Neuro-Symbolic Embodied Intelligence.

fixed rule text with the tool tiers and one generic chain (Appendix B; the appendices are a separate PDF in the code release), so no retrieval is not a blank prompt.

We make three contributions, each tied to the component and experiment that supports it.

1. **A diagnosis.** Without retrieval, five of the seven models we test pass only 10–30% of two-chain (L2) tasks against 45–95% of single-material (L1) tasks, and 74–95% of their failing tasks carry a prerequisite that is never acquired (no retrieval; Table 1, Appendix T.1). Two open-weight models do not show the gap, and we have no tested explanation for why.
2. **A decomposition of the remedy.** Same-category demonstrations improve all five affected models, the 8B one included (Table 1). On the ablation model, random demonstrations recover most of the gain, and ranking them with SAER (State-Aware Example Retrieval), a training-free scorer over the recipe graph, adds a smaller increment that our tests do not confirm (demonstration retrieval, with SAER's ranking the one step that reads the graph; Table 2, Appendix O).
3. **A measurement of defect inheritance.** Planners copy what their demonstrations contain, including an unregistered command and short quantities, at rates near zero without retrieval (§6.1, Appendix U). This is the correctness facet's mechanism.

Each effect is reported under the pre-specified scorer and under a stricter one that also charges consumed materials. The existence effect survives both; the relevance increment survives neither the pre-specified correction nor the stricter scorer.

2 Related Work

LLM agents and retrieval in Minecraft. Minecraft is an established testbed for open-ended LLM agents [Fan et al., 2022]: lifelong learners (Voyager [Wang et al., 2024a]) and open-world multi-task planners (DEPS [Wang et al., 2023], GITM [Zhu et al., 2023], JARVIS-1 [Wang et al., 2025b]), all dependent on frontier-scale models. We build on MINDcraft [White et al., 2025]. Closest to our mechanism, Leung et al. [2026] propose Goal-Oriented Graphs (GoGs), which recursively retrieve a goal's prerequisites from a dependency graph; Plan4MC [Yuan et al., 2023] traverses a pre-built skill prerequisite graph over RL skills; Optimus-1 [Li et al., 2024] retrieves world knowledge from a hierarchical knowledge graph; and Plancraft [Dagan et al., 2025] is a crafting-GUI benchmark, with an oracle planner, on which LLMs struggle with multi-step crafting. Unlike these, we retrieve few-shot *demonstrations* to condition plan generation, decomposing *which* facet of knowledge provision binds performance.

Retrieval-augmented in-context learning. In-context learning (ICL) [Brown et al., 2020] adapts LLMs from demonstrations without parameter updates; Min et al. [2022] show that on classification tasks label correctness matters less than label space, input distribution and format. Retrieval-based ICL selects examples per query: unsupervised methods (BM25 [Robertson and Zaragoza, 2009], SBERT [Reimers and Gurevych, 2019]) use lexical/semantic similarity; EPR [Rubin et al., 2022] and LLM-R [Wang et al., 2024b] train retrievers from LLM feedback; InstructRAG [Wang et al., 2025a] retrieves over an instruction graph; broader RAG [Lewis et al., 2020, Gao et al., 2023] augments facts rather than task demonstrations. SAER is *training-free* (unlike EPR/LLM-R) and reads material-tier similarity off the `minecraft-data` crafting graph, selecting demonstrations rather than subgoals with no traversal at inference. BM25 is our lexical baseline (§4.2); SBERT appears in Appendix L.

Planning beyond Minecraft and across scales. Embodied planning beyond Minecraft [Huang et al., 2022] has explored complementary directions: SayCan [Ichter et al., 2023] grounds actions in affordances, LLM+P [Liu et al., 2023] delegates to a PDDL planner, Guan et al. [2023] have the LLM construct the PDDL domain model itself under validator feedback, and Reflexion [Shinn et al., 2023] refines plans via post-execution self-evaluation. The LLM-Modulo position [Kambhampati et al., 2024] generalises this family: LLMs as approximate knowledge sources paired with external model-based verifiers. Our validator plays that role only at runtime; the intervention we study is *pre*-generation, supplying symbolic knowledge rather than checking the output. None of these addresses how much of that knowledge must be supplied, or how the answer varies with scale. Shi et al. [2023] show irrelevant context distracts models (motivating our category filter); Liu et al. [2024] document a mid-prompt position bias. PlanBench [Valmeekam et al., 2023] argues commonsense benchmarks conflate planning with retrieval, and Xu et al. [2026] report a *latent-planning* depth

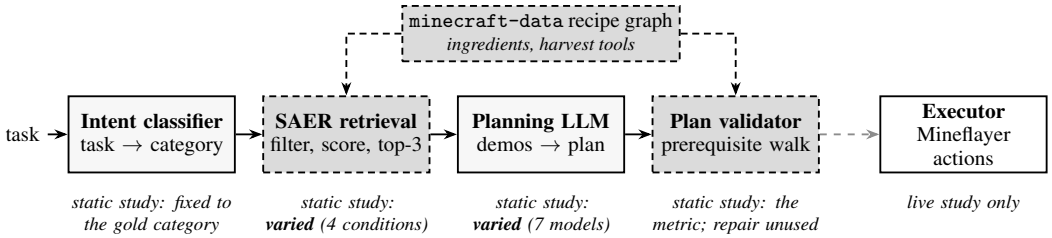


Figure 1: Pipeline, and each component's role in the confirmatory static study. Dashed, shaded boxes are symbolic and do not learn: the `minecraft-data` recipe graph and the two modules that read it. The intent classifier and the planning LLM are LLM calls. Only the retrieval condition and the planning model are varied; the task category is supplied from the benchmark label; the validator acts only as the scorer, and its repair path and the executor (grey dashed arrow) are exercised only in the live study (Appendix H).

ceiling that massive scaling leaves unresolved. We isolate another constraint — stating the rules is not enough, and the binding limit is procedural knowledge in *task-matched* form — and an exploratory thinking-mode ablation (Appendix Q) is consistent with test-time reasoning raising L1 without unlocking L2.

3 Method

3.1 System Overview

An intent classifier assigns the task a category, SAER retrieves three same-category demonstrations, the planning LLM writes a step plan, a plan validator checks its prerequisites (and at runtime may repair it), and an executor turns the steps into Mineflayer actions (Appendix C; Figure 1).

Why demonstrations rather than the graph. The recipe graph could be handed to the model directly, as Goal-Oriented Graphs [Leung et al., 2026] do, or to a classical planner, which can derive a crafting target's prerequisite chain deterministically; the release contains such a chain resolver on its runtime repair path. We study demonstrations for three reasons. The graph covers only part of the task: `minecraft-data` lists crafting recipes but no smelting route (nothing yields an iron ingot from raw iron) and no tool-tier ordering, and SAER's ingredient term is zero on 18 of the 60 tasks, every gathering and combat task among them (§3.3). A demonstration delivers the prerequisites already in order and in the bot's executable command grammar. And the agent we build on receives tasks as free-form chat across categories, for which demonstrations are the interface it already has. The LLM is therefore our object of study, not a competitor to a classical planner. Supplying each target's resolved chain is the natural next baseline, and we have not run it.

3.2 Category Filtering

The intent classifier (`getIntent()` in the release) asks the planning LLM for one label from a fixed set: `craft`, `collect`, `cook`, `build`, `combat` and `explore`, plus a `general` fallback. `combat` was added *after* the evaluation runs, when we found the original classifier could not route to the combat file (Appendix V). Only demonstrations from the file mapped to the label are admitted as retrieval candidates. The filter is motivated by the finding that irrelevant context degrades reasoning [Shi et al., 2023], not measured against it: no condition here runs unfiltered. In the static evaluation the category is the task's ground-truth label, so retrieval quality is an *oracle-category upper bound*; the bound is tight here, since the deployed classifier routes 180/180 trials to the correct file, on one provider endpoint and 60 templated prompts (Appendix V).

3.3 Composite Scoring

Given a query task q and a candidate example e from the same category, SAER computes a scalar relevance score $S(q, e) \in [0, 1]$:

$$S(q,e) = \alpha \cdot I(q,e) + \beta \cdot K(q,e) + \gamma \cdot M(q,e) + \delta \cdot L(q,e), \tag{1}$$

where $\alpha{+}\beta{+}\gamma{+}\delta{=}1$. The benchmark uses the *cold-start* weights $(0.50, 0.35, 0.00, 0.15)$ throughout, for an empty inventory; a *default* $(0.40, 0.30, 0.20, 0.10)$ for a known inventory targets mid-task replanning and is not evaluated here. Cold start sets $\gamma{=}0$ because M never depends on q, so it would act as a standing preference for low-tier examples rather than a relevance term. The default's 4:3:2:1 ordering encodes our prior on each dimension's discriminative value (ingredient overlap first, level only a tiebreaker). Weights were fixed before evaluation and no experiment here shows them optimal: on the frozen benchmark the deployed weights are indistinguishable from uniform (Appendix K), and an earlier 35-task ablation is silent on their ordering (Appendix J). They are a working point, not a demonstrated optimum.

All four components take values in $[0, 1]$; the first two are Jaccard overlaps, the last two normalised proximities. **Ingredient Overlap (I).** Let R_q, R_e be the crafting ingredient sets of q and e, resolved from the `minecraft-data` recipe graph: $I(q, e) = |R_q \cap R_e|/(|R_q \cup R_e| + \varepsilon)$, $\varepsilon = 10^{-6}$; I is identically zero when either side resolves to no recipe, which measurement puts at all 16 gathering and combat tasks plus 2 of the 44 crafting ones; on those 18 the ranking falls to K and L alone. SAER's increment over BM25 is concentrated in the complement, where $\alpha{\cdot}I$ can fire (Appendix O). **Keyword Overlap (K).** Let W_q, W_e be content-word sets (nouns, verbs, item names after stop-word removal): $K(q, e) = |W_q \cap W_e|/(|W_q \cup W_e| + \varepsilon)$. **Mining-Tier Gap ($M$).** With $t_{\text{req}}(e) \in \{0, 1, 2, 3\}$ the minimum pickaxe tier e requires (0=none, 1=wooden, 2=stone, 3=iron) and t_{player} the agent's best tier, $M = 1 - \max(0,\, t_{\text{req}}(e) - t_{\text{player}})/3$; with $\gamma{=}0$ it enters no reported number. **Level Proximity (L).** Examples carry a hand-annotated $\ell_e \in \{1, 2, 3\}$; the query's $\ell_q \in \{1, 2, 3, 4\}$ comes from surface keywords in the request (default 2), and $L = 1 - |\ell_q - \ell_e|/3$. The heuristic never reads the benchmark's tier label and does not recover it — it agrees on 20/60 tasks, what a constant "L2" guess scores — so $\delta{\cdot}L$ carries no tier oracle (Appendix V). How the ranked examples enter the prompt, and why no term in that construction is learned, is stated in Appendix B.

3.4 Context Assembly

The top-k candidates ($k{=}3$) are ordered by descending $S(q, e)$, so the most relevant comes first; the block itself sits mid-prompt, after the command reference. Each example is rendered as plain text, not JSON: `Step N:`, `Prerequisites:`, `Produces:`, then one `Command:` per bot call; the model is asked for the same shape with `Reasoning:` replacing `Produces:` — a step-by-step form after Wei et al. [2022] and ReAct [Yao et al., 2023].

3.5 Plan Validation

The plan validator (`PlanValidator` in the release) checks a plan's prerequisite structure over a virtual inventory before any in-game action. At runtime, on failure, it permits one gap-repair regeneration before execution (Appendix T). In the static study its walk is reimplemented single-pass and repair-free as the scorer (§4.3), so the repair path is exercised only by the live study (Appendix H), and every static number, Table 1 and the ablation included, measures the unrepaired plan.

4 Experiments

4.1 Evaluation Tasks

We evaluate on a held-out benchmark of **60 tasks** spanning three Minecraft task categories (44 crafting, 11 gathering, 5 combat) and three difficulty tiers. No task appears in the demonstration set, verified by slug-level non-overlap checking after an earlier benchmark revision replaced 24 overlapping tasks. The benchmark and training data were frozen before any model evaluation began, and no task was added or removed after seeing model outputs. Residual structural overlap is modest. For the 44 crafting tasks the symmetric ingredient Jaccard to the nearest demonstration has mean 0.188 (0.290 on a later corrected bank) and never exceeds 0.333 on the bank the runs retrieved over; no gathering task's goal item is the final product of any demonstration, whereas the same check flags all five combat tasks (§4.4). On the evaluation-time bank the gain rises with overlap, but the direction reverses on the corrected bank, and the high-overlap stratum is the iron- and stone-tier multi-hop chains, so overlap is substantially a proxy for prerequisite depth and neither panel points to near-copies (Appendix M).

L1 (20 tasks): single-material tasks achievable with basic tools (`wooden_sword`, `chest`, `bowl`). **L2** (20 tasks): two-chain tasks typically culminating in iron-tool use (`iron_axe`, `cauldron`, `hopper`); L2 is the primary discrimination tier because it requires the complete wooden→stone→iron pickaxe chain. **L3** (20 tasks): three-or-more-chain tasks involving deep dependency trees (`compass`, `golden_pickaxe`, `anvil`).

4.2 Models and Conditions

We evaluate seven models from five organisations (Appendix D): five primary-study models under `no_retrieval` and `saer_cold`, and a two-model open-weight replication panel (Gemma4-26B/31B) outside the pre-specified test family. Four retrieval conditions are compared on Qwen2.5-Max, the ablation model. Sampling temperature is 0.7 for Qwen3-8B, GPT-4o and Claude Sonnet 4.5, the value in the harness's evaluation profiles, not tuned on this benchmark. The Qwen2.5-Max, DeepSeek-V3 and Gemma4 requests set no temperature, so those endpoints used their server defaults, which the run files do not record (Appendix D). Every within-model comparison, and so every retrieval effect, keeps the model and its setting fixed; levels across models, and the three cross-model tests of the family, mix settings.

Conditions.

- **no_retrieval**: No retrieved examples; the task and the shared rule text only (Appendix B).
- **saer_cold**: SAER with cold-start weights $(\alpha, \beta, \gamma, \delta)=(0.50, 0.35, 0.00, 0.15)$; agent inventory treated as empty.
- **random_retrieval** (Qwen2.5-Max ablation only): Three examples selected uniformly at random from the same-category training set.
- **bm25_retrieval** (Qwen2.5-Max ablation *and* evaluated cross-model on all five primary-study models, §5.2): Okapi BM25 [Robertson and Zaragoza, 2009] (k_1=1.5, b_L=0.75; b_L is BM25's length-normalisation term, not McNemar's b).

4.3 Evaluation Protocol

Static plan evaluation. For each task and condition the planning LLM writes a plan, which is scored offline without a live bot: the validator walks the plan's commands (`collectBlocks`, `craftRecipe`, `smeltItem`) over a virtual inventory that starts empty (pseudocode in Appendix T). Each condition is run with three seeds; a task passes if it passes in at least two (**majority vote**).

Metrics. The primary metric is the *Strict Plan Pass Rate* (SPR; labelled TCR, for task completion rate, in the released scripts[1]): the fraction of tasks whose plan has no prerequisite gap in the inventory walk and ends with the goal item present. SPR measures the prerequisite-structure validity of a plan, not task completion. *Plan Validity Rate* (PVR) checks only *explicitly declared* preconditions and the goal item, not implicit tool-tier requirements, so the gap PVR−SPR separates structural failures from detail-level ones.

Two scorings. The pre-specified scorer credits each produced item but never debits consumed ones, on the rationale that quantities are the executor's job; an item produced once satisfies every later requirement. We therefore also report a *debited* SPR, which charges each craft and smelt so that every produced unit is spent once (Appendix U). The leniency is not symmetric: on Qwen2.5-Max, debiting removes 0 of `no_retrieval`'s 19 passes but 7 of random's 38, 14 of BM25's 42 and 16 of SAER's 49, because a scorer that never debits can only over-credit plans that produce things, and retrieval makes plans produce more. Credit-only remains the confirmatory scorer, since the test family was fixed for it; debited figures appear beside it in Tables 1 and 2. Against the published Plancraft oracle the scorer agrees on 95.8% of 11,177 craft invocations (κ=0.753). That certifies single-craft feasibility only: one issue class of six, not the tool-gap class behind two models' failures (Appendix A).

Re-validation. Two harness defects were found after generation: a missing recipe-resolver hook that disabled three of SAER's scoring dimensions, and a missing mining-drop alias in the scorer. Every `saer_cold` arm in Table 1 was regenerated after the fix, so each is a later run than its `no_retrieval` arm, and every stored seed plan of every condition was then re-scored with one fixed validator. All

[1] Appendices, code and data: `https://github.com/Jayden23018/existence-before-relevance`

Table 1: Cross-architecture results (60 tasks, 3-seed majority vote). SPR under the pre-specified credit-only scorer and under debiting (§4.3); PVR and the L1–L3 tier counts (of 20 each) are credit-only. **L2** is the discrimination tier. Gap = PVR−SPR in tasks. Δ = `saer_cold` minus `no_ret` (`no_retrieval`), differencing the rounded percentages, with McNemar exact p; **bold** = below α'=0.0038. ‡ = 4B active parameters (Mixture-of-Experts). The replication panel lies outside the pre-specified family. Verdicts in Appendix O and Appendix U; run provenance in Appendix N; denominator sensitivity in Appendix F.

		SPR (/60)							SAER Δ (pp)	
Model	Cond.	credit	debited	PVR	Gap	L1	**L2**	L3	credit	debited
Qwen3-8B	no_ret	14 (23%)	13 (22%)	47	+33	9	3	2	—	—
Qwen3-8B	saer_cold	26 (43%)	24 (40%)	48	+22	12	11	3	+20 (p=0.0075)	+18 (p=0.0127)
GPT-4o	no_ret	23 (38%)	20 (33%)	29	+6	17	3	3	—	—
GPT-4o	saer_cold	41 (68%)	34 (57%)	46	+5	17	16	8	**+30** (p<0.001)	**+24** (p<0.001)
Claude S. 4.5	no_ret	25 (42%)	22 (37%)	29	+4	19	4	2	—	—
Claude S. 4.5	saer_cold	51 (85%)	35 (58%)	57	+6	20	13	18	**+43** (p<0.001)	**+21** (p<0.001)
Qwen2.5-Max	no_ret	19 (32%)	19 (32%)	28	+9	17	2	0	—	—
Qwen2.5-Max	saer_cold	49 (82%)	33 (55%)	56	+7	20	14	15	**+50** (p<0.001)	**+23** (p<0.001)
DeepSeek-V3	no_ret	33 (55%)	25 (42%)	41	+8	17	6	10	—	—
DeepSeek-V3	saer_cold	52 (87%)	35 (58%)	55	+3	19	17	16	**+32** (p<0.001)	+16 (p=0.0309)
Replication panel (outside the pre-specified Bonferroni family)										
Gemma4-26B‡	no_ret	44 (73%)	35 (58%)	53	+9	18	12	14	—	—
Gemma4-26B‡	saer_cold	48 (80%)	28 (47%)	58	+10	17	16	15	+7 (p=0.388)	−11 (p=0.118)
Gemma4-31B	no_ret	46 (77%)	37 (62%)	53	+7	18	13	15	—	—
Gemma4-31B	saer_cold	53 (88%)	37 (62%)	58	+5	19	17	17	+11 (p=0.0156)	0 (p=1.000)

confirmatory SPR, PVR and McNemar values come from that pass (Appendix N); three exploratory panels quote runtime verdicts beside re-scored ones (Appendix S, Appendix R, Appendix Q).

Statistical testing. McNemar's exact binomial test [McNemar, 1947] on per-task majority-vote outcomes, α=0.05, Bonferroni-corrected over 13 pre-specified tests (5 per-model + 5 ablation + 3 cross-model): α'=0.0038. We call a result *family-wise significant* if $p<\alpha'$, *nominal* if $0.0038 \leq p < 0.05$, and *not significant* otherwise; every headline verdict is a Bonferroni verdict. The uniformly more powerful Holm step-down [Holm, 1979] over the same 13 tests is reported post hoc as a labelled sensitivity analysis (Holm rejects 11, Bonferroni 9; Appendix O). The Gemma4 panel lies outside the family and is reported descriptively. The 55 demonstrations (25 crafting, 21 collecting, 9 combat) were AI-drafted from hand-written seeds and hand-checked (Appendix E).

4.4 Combat Subgroup

The five combat tasks (L1–L3) are part of the 60 and counted in Table 1, so they form a within-benchmark subgroup, not a hold-out. Five of the nine combat demonstrations target the same five mobs and differ only by slug prefix (`combat/creeper` vs. `combat/kill_creeper`), which the slug-level check misses; the runs behind Table 1 retrieve over them (Appendix O bounds what that costs). A separate probe re-runs the five tasks on Qwen2.5-Max under all four conditions without those five demonstrations, over a combat pool with acquisition chains corrected for a cold start (Appendix S).

5 Results

5.1 Main Results

Finding 1: an L2 bottleneck in five of seven models. The five primary-study models score 10–30% on L2 without retrieval (median 15%; a 2×5 homogeneity test does not reject, p=0.539) against 45–95% on L1 (median 85%). The Gemma4 replication panel already clears 60–65% on L2 without retrieval, so the diagnosis applies to the five primary models, not to every model we ran. We have no tested explanation for the difference.

Finding 2: retrieval helps all five, the 8B model included. SAER improves all five primary models by $+20$ to $+50$ pp, four family-wise significantly; Qwen3-8B gains least ($+20$ pp, p=0.0075: Holm yes, Bonferroni no). Under debiting all five gains stay positive ($+16$ to $+24$ pp) and three clear α'. Gain does not track parameter count (DeepSeek-V3, 671B, gains less than Qwen2.5-Max), and two of five sizes are undisclosed and a third only approximate (Appendix D), so we make no capacity-scaling claim. Qwen3-8B keeps the largest PVR$-$SPR gap ($+22$ vs. $+3$ to $+7$): given correct templates it still mis-specifies counts (2 iron ingots for a helmet needing 5), a residual distinct from the bottleneck. Of the five, DeepSeek-V3 alone reaches 10/20 at L3 without retrieval (others 0–3/20); with SAER it converges with Qwen2.5-Max overall (p=0.453). On the replication panel the credit-only gains ($+7$ and $+11$ pp) do not survive debiting.

The L2 tier. L2 recovery spans $+40$ to $+65$ pp, again non-monotone in parameter count. On Qwen2.5-Max random retrieval alone lifts L2 from 10% to 70%, and `saer_cold` reaches the same 14/20, so L2 does not separate SAER from random; BM25 trails both there (11/20) but leads random at L3 (14/20 vs. 7/20). As BM25 and random do not differ overall (p=0.424), we read neither tier split as more than noise (per-tier counts in Appendix P).

5.2 Qwen2.5-Max Four-Condition Ablation

Table 2: Four-condition ablation on Qwen2.5-Max (60 tasks, 3-seed majority vote) under both scorings. Δ is against `no_retrieval`, or between the two named arms, differencing rounded percentages; 95% CI on the paired difference [Newcombe, 1998], credit-only. b/c = tasks passed only by the condition (or first-named arm) / only by the comparator, with McNemar exact p.

	Credit-only (pre-specified)				Debited		
Condition	SPR	PVR	Δ [95% CI]	b/c (p)	SPR	Δ	b/c (p)
no_retrieval	19	28	—	—	19	—	—
random	38	54	+31 [19, 42]	19/0 (<0.001)	31	+20	13/1 (0.0018)
BM25	42	53	+38 [24, 50]	24/1 (<0.001)	28	+15	12/3 (0.0352)
saer_cold	49	56	+50 [36, 61]	30/0 (<0.001)	33	+23	15/1 (<0.001)
BM25 vs. random			+7 [−5, 18]	9/5 (0.424)		−5	1/4 (0.375)
SAER vs. BM25			+12 [3, 20]	7/0 (0.0156)		+8	6/1 (0.125)

(1) **Existence is the primary lever.** The step from no retrieval to random same-category demonstrations is the largest one ($+31$ pp; $+20$ pp debited, b=13, c=1, p=0.0018), family-wise significant under both scorings. The step adds same-category worked examples to a rule text that already states the output format and one generic chain. Random demonstrations carry the same step grammar as SAER's (§3.4), so part of this may be exposure to format rather than to procedural content, in line with format effects on classification tasks [Min et al., 2022]; no condition here separates the two. The combat subgroup agrees: re-run without its five near-duplicate demonstrations, no retrieval reaches 0/5 and every retrieval condition 3/5 or better, too few tasks to rank the variants (Appendix I).

(2) **Relevance is directional, not established.** SAER over BM25 gives b=7, c=0, p=0.0156, $+12$ pp [95% CI $+3$, $+20$]. Four qualifications bound it. *Power:* with seven discordant tasks the smallest attainable two-sided p is $2 \cdot 0.5^7$=0.0156, so the Bonferroni bar (which needs $b+c \geq 10$) was out of reach whatever the outcome; post hoc Holm clears it at the pre-specified family size m=13 and fails at m=14 (Appendix O). *Scorer:* debiting reduces it to b=6, c=1, p=0.125, and BM25 (28) falls below random (31). *Generation epoch:* Qwen2.5-Max's BM25 arm was generated 16 days before its SAER arm on a provider alias, not a date-pinned identifier, so an endpoint change is a rival explanation that re-scoring cannot remove (Appendix N). *Replication:* across the five primary models SAER beats BM25 on four (nominally on two) and trails by one task on GPT-4o; the one-sided 4–1 sign test gives p=0.188, and six of the seven discordant tasks lie in the 42 where ingredient overlap is non-zero (Appendix O). One check cuts the other way: SAER is handed the *more* defective demonstrations (top-1 materially short on 22/60 tasks vs. BM25's 8/60, p=0.00012) and still scores higher, so demonstration quality does not explain its edge (Appendix U). We read relevance as a consistent but unconfirmed increment.

6 Analysis and Discussion

6.1 Why L2 Fails, and What Demonstrations Transmit

The L2 wall holds with the tool chain stated in the rule text, and it does not track scale. Counting the scorer's typed issues over every no-retrieval failure (Appendix T.1), 20–41 of each primary model's 27–46 failing tasks (74–95%) carry an *unacquired prerequisite*: a declared precondition, a craft ingredient or a tool never obtained. The signature is shared, its composition is not: ingredient gaps run 39 tasks on Qwen3-8B against 3 on GPT-4o, and tool gaps 33 on Qwen2.5-Max against 2 on Claude Sonnet 4.5, so we claim no common failure profile, and the version-specific naming check fires on no failure in the primary study (the rule text states the drop names). Correctness binds on one exploratory contrast: on a superseded 35-task benchmark six L2 demonstrations declared a stone pickaxe they never crafted, and under one scorer SAER read 1/13 at L2 against random's 6/13; the post-repair reading (11–12/13) is confounded with two concurrent validator fixes, and the episode predates every run reported here (Appendix J).

Defects are inherited verbatim. The mechanism behind that contrast is measurable on the frozen runs. All nine combat demonstrations use `!equipItem`, which is not in the bot's command registry (`!equip` is). Across every stored combat-task plan, superseded runs included, the string appears in 0 of 505 trials without retrieval and in 726 of 1,016 with it (71.5%; 72.9% under SAER, 74.2% BM25, 61.2% random): the models never invent the command and copy it about three times in four once shown it (Appendix V). Quantities travel the same way. Nine of the collecting bank's 17 `oak_planks` calls ask for 4, too few for a crafting table, sticks and a pickaxe. On the 11 gathering tasks four of the five models never emit 4 without retrieval (the rule text's chain shows 12), yet under SAER they do in 10 of 12 trials on Qwen2.5-Max, 4 of 15 on DeepSeek-V3 and 2 of 17 on GPT-4o, while Claude Sonnet 4.5 overrides the demonstration every time (Qwen3-8B emits 4 under both conditions; Appendix U). Neither defect moves a credit-only number, since that scorer credits no equip step and charges no quantity, which is why both survived to be found.

6.2 Limitations

Scope. Beyond the offline scope stated in §1, the live study (126 trials on DeepSeek-V3) floors both conditions at 2/21 L2 trials because execution-framework failures dominate, so it neither corroborates nor contradicts the static result (Appendix H). The cross-agent study (Appendix W) probes a different retrieval stage and carries no claim. **Metric.** Credit-only SPR over-credits the retrieval arms more than the baseline (§4.3); scoring is automated and unblinded; and McNemar on majority-vote outcomes models neither task difficulty nor model$\times$condition interaction.

Relevance evidence. One pre-specified test on one model, seven discordant tasks and a 16-day generation gap on a provider alias; settling it needs a same-window re-run, which we have not done. The category filter is in every retrieval condition, so its own contribution is unmeasured. **Controls not run.** We ablate retrieval *policy*, not demonstration *surface form*: format-only and content-masked controls, and supplying each target's resolved chain, are future work.

Models and decoding. Temperature differs across models (§4.2); two primary models used date-pinned identifiers, and the other three are provider aliases, so endpoint updates between their arms cannot be excluded. Three further 8–12B models, run at provider defaults, keep SAER ceilings at 7/26/34 of 60 when re-scored (7/23/33 at runtime), below the four frontier primary models (41–52/60) but not all below Qwen3-8B's 26/60, so we claim no capacity threshold (Appendix R). Without `minecraft-data` SAER keeps only keyword and level, as on 18/60 tasks already (Appendix O).

7 Conclusion

Five of the seven LLM planners we tested pass only 10–30% of two-chain tasks without retrieval; in 74–95% of those failures a prerequisite is never acquired (§6.1). Given a shared rule text, the primary lever is the *existence* of a same-category demonstration (+31 pp on Qwen2.5-Max; +20 pp debited). *Relevance* adds an increment that neither its discordance budget nor the stricter scorer confirms, concentrated in the 42 tasks with ingredient overlap; on *capacity*, the 8B model gains too. Demonstrations are inherited, errors included, so their *correctness* is the next lever, a natural target for verifiable-reward RL [Zeng et al., 2026].

Acknowledgments and Disclosure of Funding

This work received no external funding. The authors declare no competing interests.

References

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

Gautier Dagan, Frank Keller, and Alex Lascarides. Plancraft: An evaluation dataset for planning with LLM agents. In *Proceedings of the 2nd Conference on Language Modeling (COLM)*, 2025.

Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. MineDojo: Building open-ended embodied agents with internet-scale knowledge. In *Advances in Neural Information Processing Systems*, volume 35, pages 18343–18362, 2022.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. In *Advances in Neural Information Processing Systems*, volume 36, pages 79081–79094, 2023.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pages 9118–9147, 2022.

Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, et al. Do as I can, not as I say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318, 2023.

Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Paul Saldyt, and Anil B. Murthy. Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 22895–22907, 2024.

Jonathan Leung, Yongjie Wang, and Zhiqi Shen. From entity-centric to goal-oriented graphs: Enhancing LLM knowledge retrieval in Minecraft. *Knowledge-Based Systems*, 341:115706, 2026.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

Zaijing Li, Yuquan Xie, Rui Shao, Gongwei Chen, Dongmei Jiang, and Liqiang Nie. Optimus-1: Hybrid multimodal memory empowered agents excel in long-horizon tasks. In *Advances in Neural Information Processing Systems*, volume 37, pages 49881–49913, 2024.

Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 11048–11064, 2022.

Robert G. Newcombe. Improved confidence intervals for the difference between binomial proportions based on paired data. *Statistics in Medicine*, 17(22):2635–2650, 1998.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

Stephen E. Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Ohad Rubin, Jonathan Herzig, and Jonathan Berant. Learning to retrieve prompts for in-context learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 2655–2671, 2022.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.

Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *Advances in Neural Information Processing Systems*, volume 36, pages 38975–38987, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024a.

Liang Wang, Nan Yang, and Furu Wei. Learning to retrieve in-context examples for large language models. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 1752–1767, 2024b.

Zheng Wang, Shu Xian Teo, Jun Jie Chew, and Wei Shi. InstructRAG: Leveraging retrieval-augmented generation on instruction graphs for LLM-based task planning. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pages 1413–1422, 2025a.

Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. Describe, explain, plan and select: Interactive planning with LLMs enables open-world multi-task agents. In *Advances in Neural Information Processing Systems*, volume 36, pages 34153–34189, 2023.

Zihao Wang, Shaofei Cai, Anji Liu, Yonggang Jin, Jinbing Hou, Bowei Zhang, Haowei Lin, Zhaofeng He, Zilong Zheng, Yaodong Yang, et al. JARVIS-1: Open-world multi-task agents with memory-augmented multimodal language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(3):1894–1907, 2025b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

Isadora White, Kolby Nottingham, Ayush Maniar, Max Robinson, Hansen Lillemark, Mehul Maheshwari, Lianhui Qin, and Prithviraj Ammanabrolu. Collaborating action by action: A multi-agent LLM framework for embodied reasoning. *arXiv preprint arXiv:2504.17950*, 2025.

Yi Xu, Philipp Jettkant, and Laura Ruis. The depth ceiling: On the limits of large language models in discovering latent planning. In *Proceedings of the 3rd Conference on Language Modeling (COLM)*, 2026.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

Haoqi Yuan, Chi Zhang, Hongcheng Wang, Feiyang Xie, Penglin Cai, Hao Dong, and Zongqing Lu. Skill reinforcement learning and planning for open-world long-horizon tasks. *arXiv preprint arXiv:2303.16563*, 2023.

Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Lifan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting Li, Jacqueline He, Runlong Zhou, Tong Chen, et al. RLVE: Scaling up reinforcement learning for language models with adaptive verifiable environments. In *International Conference on Machine Learning (ICML)*, 2026.

Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei Lu, Xiaogang Wang, et al. Ghost in the Minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory. *arXiv preprint arXiv:2305.17144*, 2023.